

Лекция 11.

Почтовая инфраструктура Linux и контейнерная платформа Mailcow

Цель лекции

Понять базовый mail flow от отправителя к получателю
Разобрать роли MUA, MTA, mailbox, SMTP, Submission и IMAPS

Настроить внутренний домен company.test и FQDN mail.company.test

Развернуть Mailcow на отдельной Linux VM

Создать два mailbox и доказать доставку письма от user1 к user2

Учебные вопросы

1. MUA, MTA, mailbox; SMTP, Submission и IMAPS
2. Postfix и Dovecot; domain, mailbox и address
3. DNS A/MX и обзор SPF, DKIM, DMARC
4. Queue, SMTP response codes, Open Relay и Rspamd
5. Основные роли Mailcow
6. Backup, Restore, Update и диагностика

Главный результат лаборатории

Mail domain: company.test

FQDN сервера: mail.company.test

Mail server: 192.168.30.20

Mailbox 1: user1@company.test

Mailbox 2: user2@company.test

Почта считается работающей, когда письмо
отправлено, доставлено и прочитано

Внутренний mail flow user1 → user2 через Mailcow



Проверка

- ✓ Вход user1
- ✓ Отправка письма
- ✓ Доставка user2
- ✓ Чтение через Webmail / IMAPS



Главный вывод

Почта работает, когда письмо реально отправлено, доставлено и прочитано.

Единый сценарий лекции

Настроить DNS A и MX для внутреннего домена

Подготовить чистую VM mail.company.test

Развернуть Mailcow

Создать domain company.test и два mailbox

Отправить письмо user1 -> user2 через Submission

Прочитать письмо user2 через Webmail или IMAPS и
проверить queue

1. Почтовая система начинается с MUA, MTA и mailbox

MUA (Mail User Agent) - программа пользователя для чтения и отправки почты

MTA (Mail Transfer Agent) - серверная служба, которая принимает и передает почту

Mailbox - почтовый ящик пользователя, где хранится доставленная почта

MDA (Mail Delivery Agent) - компонент локальной доставки сообщения в mailbox

В Mailcow эти роли закрываются несколькими контейнерами и службами

Mail flow нужно проверять от отправки до чтения

MUA user1 отправляет письмо на Submission 587

Postfix принимает письмо как MTA

Rspamd оценивает сообщение

Local Delivery помещает письмо в mailbox user2

Dovecot выдает mailbox пользователю user2 по IMAPS
или Webmail

Работающий порт не доказывает, что весь mail flow
исправен

SMTP 25 и Submission 587/465 имеют разные задачи

SMTP 25 - обмен почтой между mail servers

Submission 587 - отправка письма пользователем или приложением на свой mail server

Submission обычно требует authentication и TLS

465 Submission TLS - альтернативный защищенный submission-порт

В лаборатории пользователь не отправляет обычную почту анонимно через порт 25

IMAPS и HTTPS нужны для чтения и управления

993 IMAPS - защищенный доступ клиента к mailbox

443 HTTPS - Webmail и Admin UI Mailcow

143 IMAP без TLS и POP3/POP3S можно знать обзорно

TLS защищает канал между клиентом и сервером

TLS не является end-to-end encryption содержимого письма

2. Postfix принимает и передает SMTP-почту

Postfix отвечает за SMTP-прием и SMTP-отправку

Postfix использует queue, если письмо нельзя обработать сразу

Postfix применяет маршрутизацию, ограничения и интеграцию с фильтрацией

В Mailcow Postfix работает как часть контейнерной платформы

Для Junior важно понимать роль Postfix, а не внутреннее устройство всех очередей

Dovecot дает доступ к mailbox

Dovecot отвечает за IMAP/IMAPS-доступ к почтовым ящикам

Dovecot участвует в authentication пользователей

Dovecot не следует называть просто хранилищем

Mailbox хранит письма, а Dovecot предоставляет к ним протокольный доступ

Если Dovecot отказал, письмо может быть доставлено, но пользователь не сможет его прочитать по IMAPS

Domain, mailbox и address нельзя смешивать

Domain: company.test

Mailbox: user1

Address: user1@company.test

FQDN сервера: mail.company.test

Alias - дополнительный адрес, который указывает на mailbox или группу

Ошибка в домене или адресе ломает доставку даже при работающих контейнерах

3. Внутренней лаборатории нужны А и МХ

mail.company.test. IN A 192.168.30.20

company.test. IN MX 10 mail.company.test.

А показывает, где находится mail server

МХ показывает, какой hostname принимает почту для domain

МХ должен указывать на hostname, а не прямо на IP-адрес

MX priority нужен для порядка попыток доставки

MX 10 обычно означает основной mail server

MX 20 может означать резервный mail server

Чем меньше число, тем выше приоритет

В основной лаборатории резервный MX не строится

Студент должен понимать запись MX 10

mail.company.test как основной маршрут почты
домена

.test используется только для внутреннего стенда

company.test зарезервирован для тестирования

.test не является публичным production-доменом

Лаборатория не проверяет доставку в Gmail, Outlook
или внешние домены

mail.company.test должен разрешаться внутри учебной
сети

Публичная Internet-доставляемость изучается
отдельно

PTR и IPv6 не являются целью основной лаборатории

Во внутренней лаборатории PTR для публичной доставляемости не нужен

Публичный mail server обычно требует корректный PTR публичного IP

PTR обычно настраивает владелец публичного IP-адреса

IPv6 в этой лаборатории не используется

AAAA, IPv6 PTR и FCrDNS относятся к дополнительному материалу

SPF, DKIM и DMARC объясняются без перегруза

SPF - кто может отправлять от имени domain

DKIM - цифровая подпись сообщения доменным ключом

DMARC - связывает SPF/DKIM с видимым From и задает policy

DMARC требует, чтобы SPF или DKIM прошли и были согласованы с доменом From

SPF/DKIM/DMARC полезны, но не гарантируют попадание письма во входящие

Учебные примеры DNS-аутентификации домена

company.test. IN TXT "v=spf1 mx -all"

DKIM публикуется как TXT-запись с публичным ключом

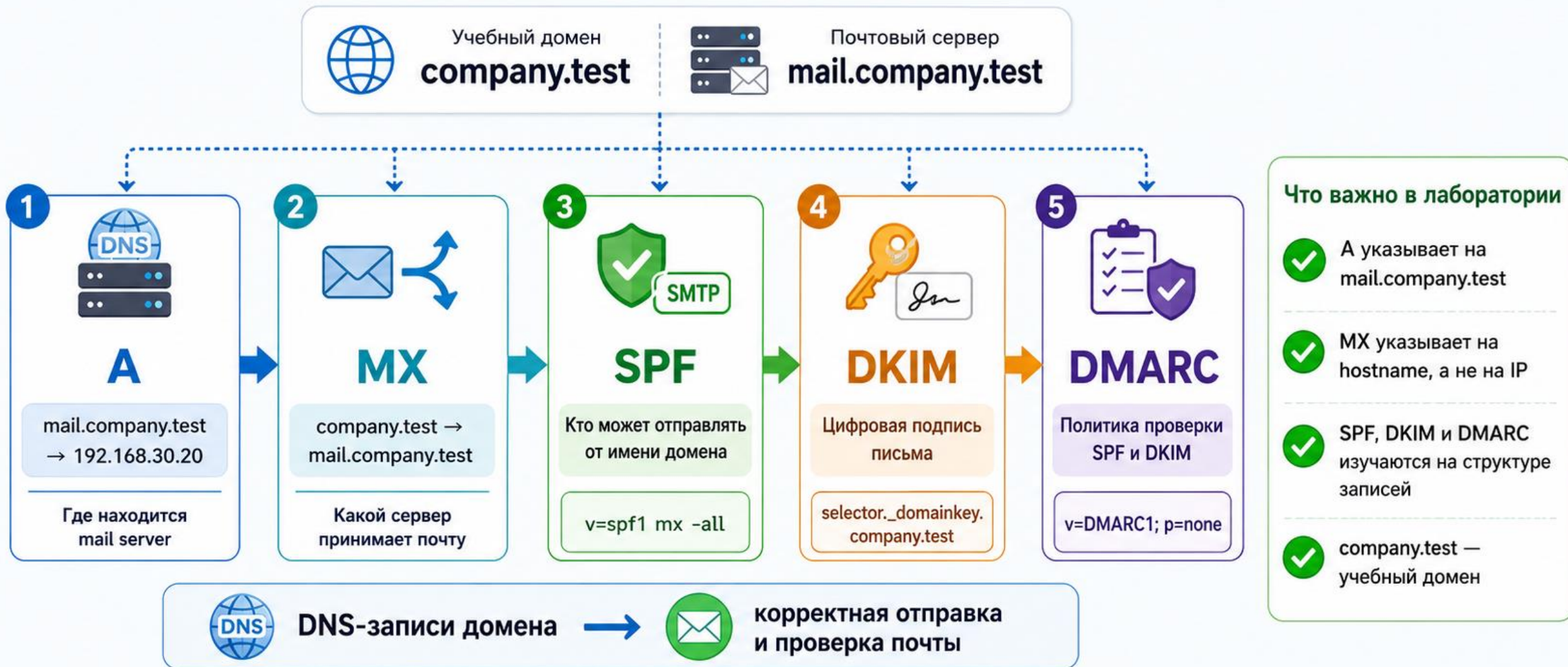
_dmarc.company.test. IN TXT "v=DMARC1; p=none;

rua=mailto:dmarc@company.test"

В реальной публичной почте значения подбирают
осторожно

SPF lookup limit, SRS и полный DMARC rollout не входят
в обязательную часть

A, MX, SPF, DKIM и DMARC в учебном домене



Главный вывод

А и MX помогают найти почтовый сервер,
а SPF, DKIM и DMARC помогают проверять подлинность отправителя.

4. Queue показывает, что происходит с письмом

Queued - письмо ожидает обработки

Deferred - временная ошибка, будет повторная попытка

Bounce - окончательная ошибка доставки

Очередь помогает отличить проблему маршрута от проблемы доступа к mailbox

Не очищать очередь до понимания причины проблемы

SMTP response codes помогают читать ошибку доставки

2xx - success

4xx - temporary error, retry later

5xx - permanent error

450 - пример временной ошибки

550 - пример постоянной ошибки

Код ответа подсказывает, ждать повторной доставки
или исправлять причину сразу

Queue commands используются осторожно

mailq - показать очередь

postqueue -p - показать очередь Postfix

postcat -q QUEUE_ID - посмотреть сообщение из queue

postqueue -f - попытаться обработать queue

postsuper -d QUEUE_ID - удалить конкретное
сообщение

Массовое удаление queue без анализа может скрыть
настоящую проблему

Open Relay нужно уметь объяснить матрицей

External -> local domain: PERMIT

Authenticated user -> external domain: PERMIT

Anonymous external -> external domain: DENY

Если anonymous external -> external domain разрешен,
сервер стал Open Relay

Open Relay быстро превращается в источник спама и
блокировок

Rspamd оценивает письмо перед доставкой

Message поступает на проверку

Rspamd применяет rules и вычисляет score

Результат может быть Allow, Spam, Quarantine или Reject

Для Junior важно понимать логику score, а не весь антиспам stack

Ошибки Rspamd можно рассматривать после базового mail flow

Mail headers помогают увидеть путь письма

Received показывает этапы прохождения письма

Authentication-Results показывает результаты

SPF/DKIM/DMARC

DKIM-Signature показывает подпись сообщения

X-Spam-* может содержать результат антиспам-проверки

Headers нужны для диагностики, когда письмо доставлено не так, как ожидалось

5. Mailcow объединяет почтовые роли в контейнерную платформу

Postfix - SMTP, прием, отправка и queue

Dovecot - IMAP/IMAPS и mailbox access

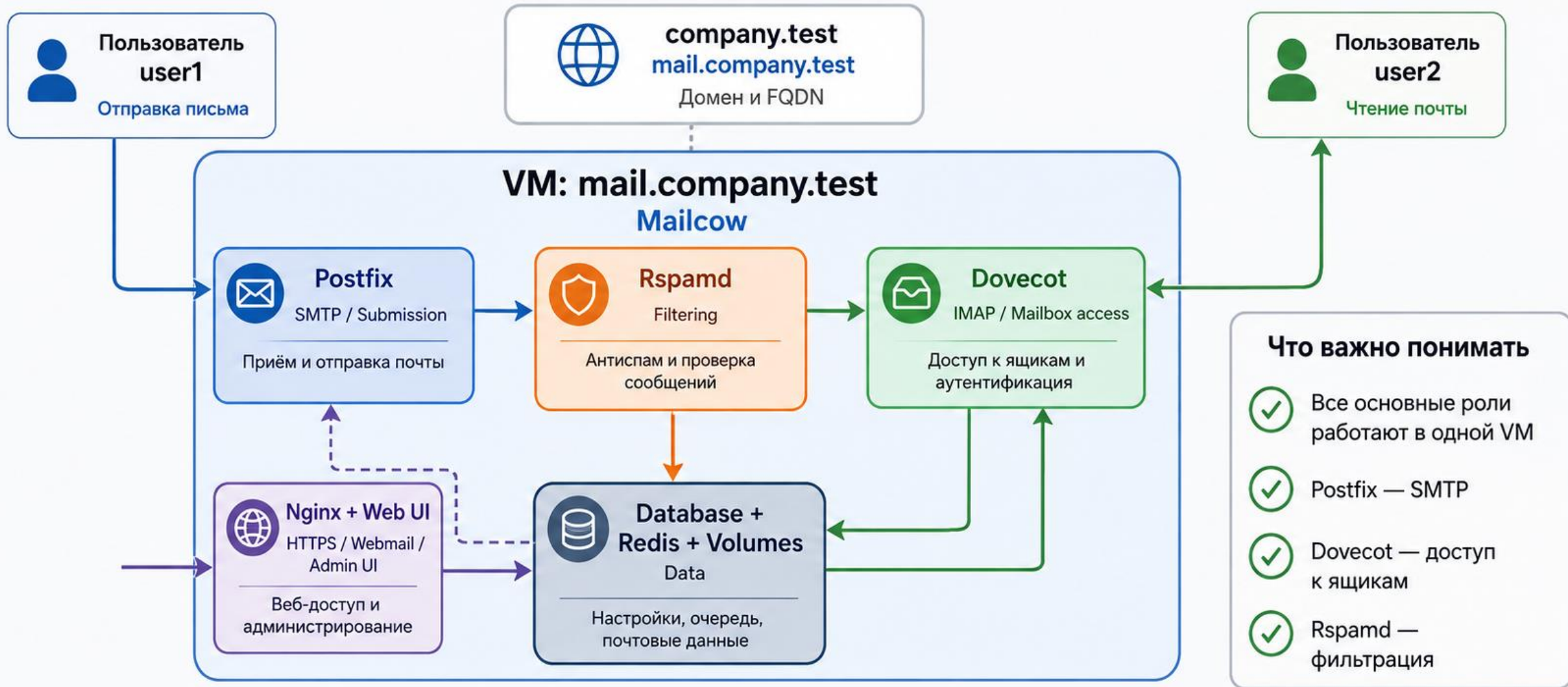
Rspamd - filtering

Nginx / Web UI - HTTPS, Webmail и Admin UI

Database, Redis и Volumes - данные, состояние и служебная информация

Junior не должен заучивать полный список всех контейнеров Mailcow

Основные роли Mailcow в одной VM



Главный вывод

Mailcow объединяет основные роли почтовой системы в одной VM:
приём, фильтрацию, хранение и доступ к почте.

Mailcow разворачивают на отдельной чистой VM

`sudo ss -ltnp` - проверить занятые порты

`docker ps` - проверить запущенные контейнеры

`systemctl --type=service --state=running` - посмотреть активные службы

Не должно быть конфликтующих Postfix, Dovecot, Nginx, Apache или других Docker projects

Чистая VM снижает количество случайных ошибок в лаборатории

FQDN должен быть выбран один раз и использоваться везде

hostname: mail

FQDN: mail.company.test

Mail Domain: company.test

Web: <https://mail.company.test>

IP: 192.168.30.20

Нельзя смешивать mail.company.test, mail.local и другие имена в одной лаборатории

Docker уже изучался, поэтому повторяем только нужное

`docker compose ps` - состояние containers

`docker compose logs` - диагностика logs

`docker exec` - временная диагностика внутри container

Постоянную конфигурацию внутри container вручную не изменяют

Mailcow управляется штатными файлами и скриптами проекта, а не ручной правкой контейнеров

Создание domain и mailbox подтверждается отправкой письма

В Mailcow Admin UI создать domain company.test

Создать mailbox user1@company.test

Создать mailbox user2@company.test

Проверить вход в Webmail для обоих пользователей

Отправить письмо user1 -> user2 и открыть его в mailbox user2

6. Backup Mailcow выполняется штатным механизмом версии

```
cd /opt/mailcow-dockerized
```

```
MAILCOW_BACKUP_LOCATION=/mnt/mailcow-backup
```

```
./helper-scripts/backup_and_restore.sh backup all
```

Перед лабораторией сверить синтаксис с

официальной документацией установленной версии

Mailcow

Active Docker volumes не копируют вручную как

универсальный backup

Backup должен быть повторяемым и проверяемым

Бэкап должен включать не только письма

mailboxes

database

configuration

certificates

DKIM keys

required secrets

Бэкап содержит sensitive data и должен быть защищен

Restore проверяет, что backup действительно полезен

Clean VM

Install compatible Mailcow

Start empty Mailcow

Run restore: `./helper-scripts/backup_and_restore.sh`

restore

Проверить Web UI, mailboxes, Submission, IMAPS, queue и Mail Flow

Backup без test restore остается предположением, а не гарантией восстановления

Update Mailcow начинается с backup

Current Version - зафиксировать текущую версию

Backup - создать резервную копию

Free Space - проверить свободное место

Update - выполнить `cd /opt/mailcow-dockerized` и
`./update.sh`

Containers - проверить состояние после обновления

Mail Flow Test - отправить и прочитать тестовое письмо

Rollback update не всегда равен запуску старых images

Старые images не гарантируют безопасный rollback

Если изменились data или database, простой откат контейнеров может не помочь

Безопаснее иметь clean deployment и restore backup

Подробные migration internals не входят в Junior-лекцию

Главный навык - не обновлять почтовую систему без backup и проверки восстановления

Диагностика Mailcow идет по уровням

Container - запущены ли нужные containers

Process - работают ли службы внутри

Port - слушают ли 25, 587, 993, 443

Protocol - проходит ли SMTP/IMAPS/TLS

Authentication - принимает ли сервер логин
пользователя

Mail Flow - письмо отправлено, доставлено и
прочитано

Диагностика почтовой системы: от контейнера до Mail Flow



Правило диагностики

Проверка идёт по цепочке: **Container → Process → Port → Protocol → Authentication → Mail Flow.**



Главный вывод

Работающий контейнер ещё не означает, что почта работает.
Почтовую систему подтверждают только успешный вход и полный **Mail Flow**.

Protocol checks подтверждают TLS и доступность портов

Submission check: openssl s_client -starttls smtp -connect mail.company.test:587 -servername mail.company.test

IMAPS check: openssl s_client -connect mail.company.test:993 -servername mail.company.test

Эти проверки показывают порт, TLS и имя сертификата

Они не доказывают полную доставку письма

Полный результат доказывает только mail flow test

swaks полезен, но остается дополнительным инструментом

swaks - удобный SMTP diagnostic client

Он помогает проверить Submission и SMTP-ответы

Не делать swaks обязательным, если пакет отсутствует

Пароль нельзя оставлять в shell history

Пароль нельзя вставлять в отчет студента открытым текстом

Главный Internal Mail Flow Test

user1@company.test отправляет письмо

Submission передает письмо в Postfix

Rspamd проверяет сообщение

Delivery помещает письмо в mailbox user2

user2@company.test открывает письмо через Webmail
или IMAPS

Студент фиксирует тему, время, отправителя,
получателя и результат доставки

Break/Fix: DNS Error и Postfix Down

DNS Error: mail.company.test не разрешается

Проверить A и MX записи для company.test

Postfix Down: SMTP/Submission нарушены

Проверить containers, logs и доступность портов

25/587

Если Submission не работает, пользователь не сможет отправить письмо

Break/Fix: Dovecot Down и Deferred Message

Dovecot Down: IMAPS нарушен, пользователь не читает mailbox

Проверить 993, logs и состояние Dovecot container

Deferred Message: mailq показывает временно отложенное письмо

Нужно найти причину Deferred, а не сразу очищать queue

Rspamd и database failures относятся к дополнительной диагностике

Monitoring не должен ограничиваться открытым портом

Host - доступность VM и ресурсы

Containers - состояние Mailcow containers

SMTP, Submission, IMAPS, HTTPS - базовые сервисы

Queue - необъяснимые Deferred

TLS Certificate - срок действия

Backup - успешное создание и контроль restore

Synthetic Mail Flow является главной проверкой

Send test mail

Submission

Delivery

Mailbox

Read mail

Measure result

TCP 25 OPEN не означает, что Mail System работает

Приемочная матрица лаборатории

hostname -f -> mail.company.test

A -> 192.168.30.20

MX -> mail.company.test

HTTPS -> Web UI работает

587 -> TLS + authentication

993 -> TLS + mailbox

user1 -> user2 -> письмо доставлено и прочитано

Queue -> нет необъяснимого Deferred

Приемочная матрица отказов и восстановления

Relay -> anonymous relay запрещен

Dovecot failure -> IMAP нарушен

Recovery -> IMAP и mail flow восстановлены

Backup -> успешно создан

Restore -> подтвержден на чистом стенде

Update -> после ./update.sh mail flow снова проверен

Дополнительный материал после базового стенда

Backup MX, PTR/FCrDNS и IPv6 mail

SPF lookup limits, SRS и полный DMARC rollout

IP reputation, domain reputation, blocklists и complaint rate

MTA-STS, DANE, S/MIME и OpenPGP

advanced Rspamd и глубокий мониторинг Mailcow
public provider SMTP limitations

Итоги лекции

Почтовая система работает только тогда, когда письмо отправлено, доставлено и прочитано

MUA, MTA, Postfix, Dovecot, mailbox и DNS выполняют разные роли

Для внутреннего стенда company.test достаточно корректных A/MX и понятного FQDN

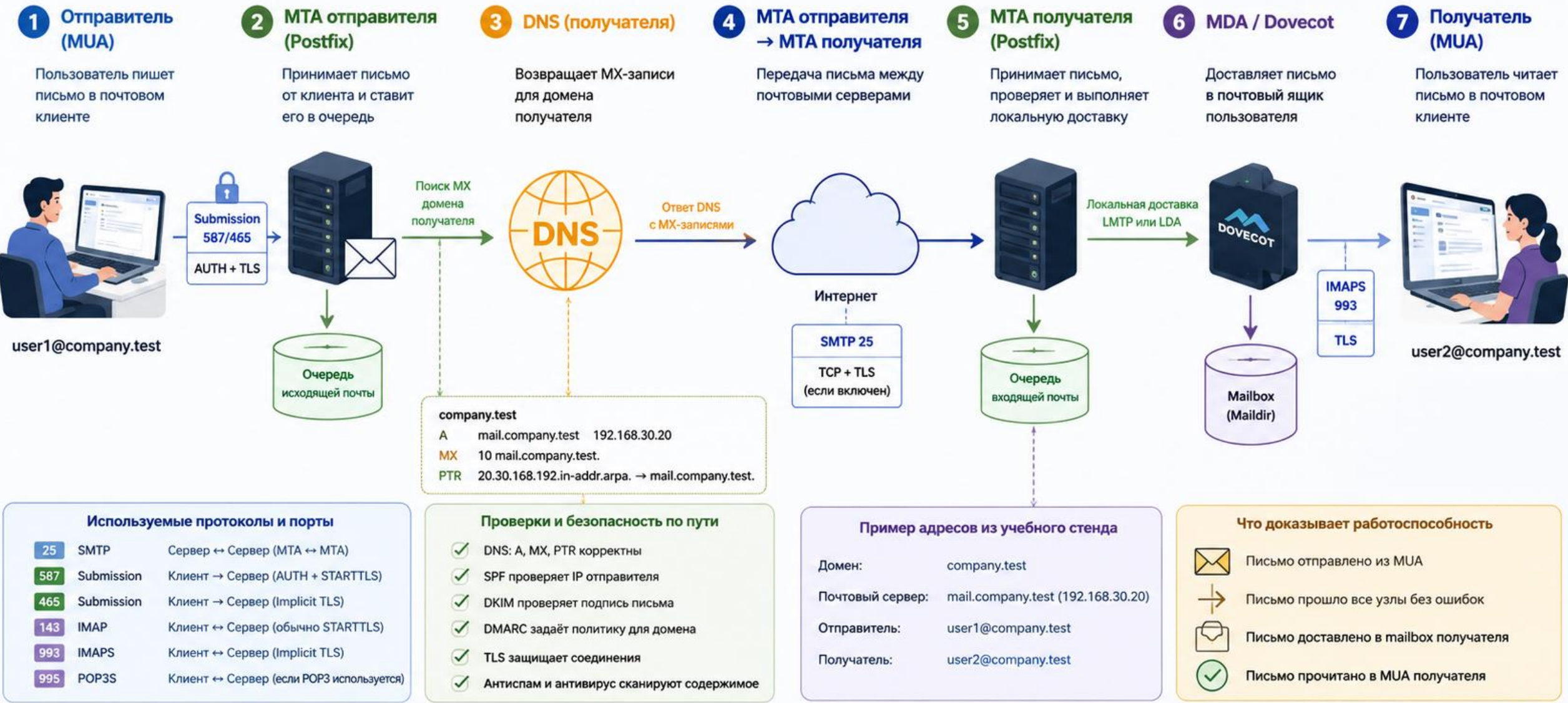
Mailcow упрощает развертывание, но не отменяет backup, restore и диагностику

Сложная публичная почтовая инфраструктура изучается после уверенного внутреннего mail flow

Контрольные вопросы

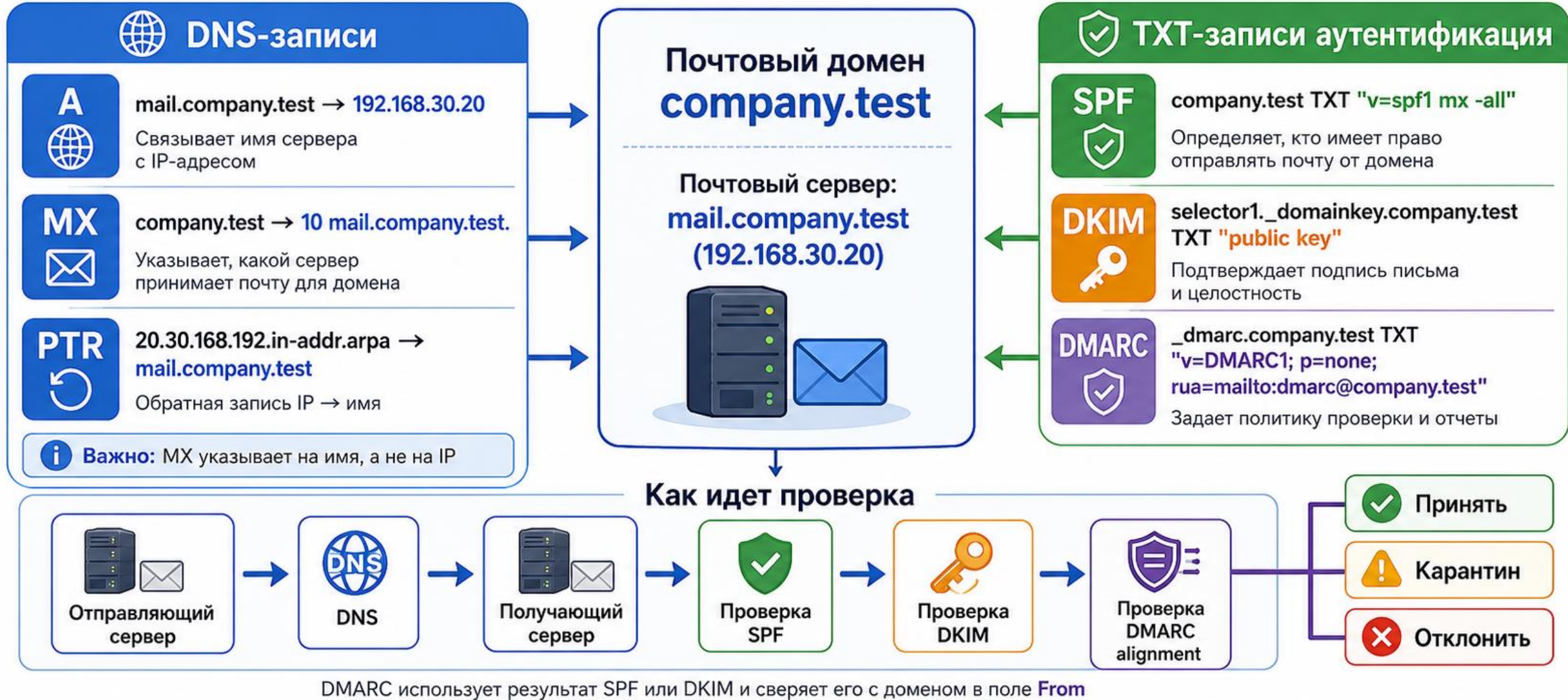
1. Чем MUA отличается от MTA?
2. Для чего используется SMTP 25?
3. Для чего используется Submission 587?
4. Для чего используется IMAPS 993?
5. Что делают Postfix и Dovecot?
6. Для чего нужны A и MX?
7. Что означает SMTP 4xx?
8. Что такое Open Relay?
9. Для чего используется Rspamd?
10. Как проверить полный Mail Flow?
11. Почему работающие Docker containers не доказывают работу почты?
12. Почему Mailcow Backup нужно проверять Restore?

Схема: полный путь письма от отправителя к получателю



💡 Главная идея: работоспособность почты подтверждается полным mail flow от отправителя до получателя через все сервисы и протоколы.

Схема: DNS-записи и аутентификация почтового домена



Главная идея

Работоспособный почтовый домен требует согласованных DNS-записей (**A**, **MX**, **PTR**) и корректной аутентификации (**SPF**, **DKIM**, **DMARC**).

Схема: основные контейнеры и роли Mailcow

Mailcow — контейнерная платформа для почтового сервера с веб-интерфейсом управления

**Безопасность**
Изоляция контейнеров,
минимальные привилегии

**Управление**
Web UI, API, автоматизация,
резервное копирование

